

Building and shipping TML4-Pipeline

This summer I worked in Prof. Mark Kon's Machine Learning Research Group at BU on TML4-Pipeline, a tool that trains a set of classifiers on a gene-expression dataset you upload and then predicts, per patient, whether a treatment is likely to work. This is a writeup of what I built.

The project started as a handful of separate prototypes of the same idea. The goal was to turn that into one tool a research group could actually use, not just the people who wrote it. My work was mostly on the parts that make that true: getting a working version standing early, putting it online, keeping it safe once it was public, building an interface a non-programmer can use, and keeping the team pointed at one design.

Getting something running early

I built one of the early end-to-end versions before the group had a merged pipeline. The point wasn't to win any benchmark, it was to have something concrete that ran start to finish so the group had a real thing to react to instead of five ideas on paper.

That early version locked in the basic shape the later work followed: a dataset comes in, gets preprocessed and turned into features, a model trains on it, the result gets scored, and a report comes out. It also fixed the app's contract with the user, which never changed after that: train a model on the data you upload, save it, load it back, predict for one patient.

The app itself

The people who most need to predict for a patient are the least likely to want to open a Python file to do it, so the whole app is built around three things you can do without touching code: train a model on an uploaded matrix, save and reload it, and predict a response for one patient.

Trained models are held in memory for your session rather than written to the server. That keeps the app simple to host, and it means one person's model never sits on disk where the next visitor could reach it. Every classifier, preprocessing option, and feature map the team adds shows up in the dropdowns automatically, so the interface stayed current instead of always trailing the code.

Putting it online

I deployed the app to Hugging Face Spaces, which is what turned it from code on a laptop into something anyone in the group can open in a browser.

That took some figuring out. A Space is configured by a YAML header at the top of the README, and the Streamlit version pinned there has to match the one in requirements. I

compared Spaces against a couple of other hosting options and priced out what running it actually costs. The thing that made the decision easy: the pipeline is plain CPU-bound scikit-learn with no GPU anywhere in it, so there's no expensive hardware to pay for and the compute tier is basically free. The instinct that this kind of ML needs a GPU turned out to be wrong, and confirming that kept hosting cheap and simple.

Security and secrets

Once an app that handles patient data is going on a public URL, secrets and how files get loaded stop being afterthoughts, so I set that up deliberately and audited the app before it went live.

The app saves models as encrypted files, locked with a key derived from a secret. I set that secret to be read from an environment variable and generated a real high-entropy key for the deployment rather than leaving anything hardcoded, kept out of version control so it never lands in the repo. That encrypted, authenticated format is also what makes loading a saved model safe: the app only ever opens files it locked itself, which closes the code-execution risk that comes with loading uploaded model files. One property worth knowing is that the key is deterministic, so changing it means every previously saved model stops opening, which makes rotating it a deliberate step rather than a free one.

The audit before launch covered the things that only matter once the app is public: confirming the secret isn't committed anywhere in the repo, that model loading can't run arbitrary uploaded code, and that trained models are held in memory per session rather than written to a shared spot on the server, so one session's work can't leak into another's.

The part that isn't code

Accepting real patient uploads on a public URL is a data question, not a code one. It's fine for the de-identified benchmark data the tool runs on now, but the moment it would touch identifiable data that becomes a compliance conversation. I'd rather that be a decision we make with Dr. Kon before launch than something we back into, so it's flagged, and the app sits behind password protection for now with a research-use framing.